

0.4.2. L'alphabet à la main, jusqu'au bout de la ligne — doublé, deux positions

La ligne A à T deux fois : en ligne 0 et en ligne 5. Écrit à la main, c'est quarante lignes — la boucle du 0.5 va l'éviter.

1. Ce qu'il faut comprendre

C'est le 0.4 et le 0.4.1 réunis : les vingt lettres en ligne 0, puis les vingt en ligne 5.

Quarante lignes presque pareilles : c'est long, et facile de se tromper. C'est exactement ce que la boucle `for` du 0.5 va raccourcir.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **975 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage	— la mise en route de la console
main()	447 o	← démarrage	— ta fonction
VBlank	8 o	← la console, à chaque image	— compte les images, 60 fois par seconde
AttendreImage	28 o	← image()	— attend l'image suivante
AttendreVBlank	16 o	← démarrage	— attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte	— attend de pouvoir écrire une case
EcrireTexte	10 o	← texte()	— écrit un texte, lettre par lettre
CopierTuiles	31 o	← démarrage	— copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	— vide les deux cartes de l'écran
Tuiles	320 o	← CopierTuiles	— 20 lettres sur 43
"A"	1 o	← texte()	— un texte gravé
"B"	1 o	← texte()	— un texte gravé
"C"	1 o	← texte()	— un texte gravé
"D"	1 o	← texte()	— un texte gravé
"E"	1 o	← texte()	— un texte gravé
"F"	1 o	← texte()	— un texte gravé
"G"	1 o	← texte()	— un texte gravé
"H"	1 o	← texte()	— un texte gravé
"I"	1 o	← texte()	— un texte gravé
"J"	1 o	← texte()	— un texte gravé
"K"	1 o	← texte()	— un texte gravé
"L"	1 o	← texte()	— un texte gravé
"M"	1 o	← texte()	— un texte gravé
"N"	1 o	← texte()	— un texte gravé
"O"	1 o	← texte()	— un texte gravé
"P"	1 o	← texte()	— un texte gravé
"Q"	1 o	← texte()	— un texte gravé
"R"	1 o	← texte()	— un texte gravé

```

"S"      1 o ← texte() – un texte gravé
"T"      1 o ← texte() – un texte gravé
"A"      1 o ← texte() – un texte gravé
"B"      1 o ← texte() – un texte gravé
"C"      1 o ← texte() – un texte gravé
"D"      1 o ← texte() – un texte gravé
"E"      1 o ← texte() – un texte gravé
"F"      1 o ← texte() – un texte gravé
"G"      1 o ← texte() – un texte gravé
"H"      1 o ← texte() – un texte gravé
"I"      1 o ← texte() – un texte gravé
"J"      1 o ← texte() – un texte gravé
"K"      1 o ← texte() – un texte gravé
"L"      1 o ← texte() – un texte gravé
"M"      1 o ← texte() – un texte gravé
"N"      1 o ← texte() – un texte gravé
"O"      1 o ← texte() – un texte gravé
"P"      1 o ← texte() – un texte gravé
"Q"      1 o ← texte() – un texte gravé
"R"      1 o ← texte() – un texte gravé
"S"      1 o ← texte() – un texte gravé
"T"      1 o ← texte() – un texte gravé

```

Lettres gravées : ABCDEFGHIJKLMNOPQRST

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```

texte() × 40    440 o sur place, dans main() + EcrireTexte (10 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <texte> // écrit un texte à l'écran
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {           // Le programme commence ici.
    // Deux lignes d'alphabet : la ligne 0, puis la même sur la ligne 5.
    // Plusieurs instructions peuvent tenir sur une même ligne :
    // chacune se termine par son « ; ».
    // 1. La ligne 0 (le 0.4) : texte(colonne, 0, "LETTRE"), colonnes 0 à 19.
    texte(0, 0, "A"); texte(1, 0, "B"); texte(2, 0, "C"); texte(3, 0, "D"); texte(4, 0, "E");
    texte(5, 0, "F"); texte(6, 0, "G"); texte(7, 0, "H"); texte(8, 0, "I"); texte(9, 0, "J");
    texte(10, 0, "K"); texte(11, 0, "L"); texte(12, 0, "M"); texte(13, 0, "N"); texte(14, 0, "O");
    texte(15, 0, "P"); texte(16, 0, "Q"); texte(17, 0, "R"); texte(18, 0, "S"); texte(19, 0, "T");
    // 2. La ligne 5 (le 0.4.1) : les mêmes lettres, le 2e nombre vaut 5.
    texte(0, 5, "A"); texte(1, 5, "B"); texte(2, 5, "C"); texte(3, 5, "D"); texte(4, 5, "E");
    texte(5, 5, "F"); texte(6, 5, "G"); texte(7, 5, "H"); texte(8, 5, "I"); texte(9, 5, "J");
    texte(10, 5, "K"); texte(11, 5, "L"); texte(12, 5, "M"); texte(13, 5, "N"); texte(14, 5, "O");
    texte(15, 5, "P"); texte(16, 5, "Q"); texte(17, 5, "R"); texte(18, 5, "S"); texte(19, 5, "T");

    while (true) {      // La boucle de jeu : recommence pour toujours...
        image();         // ... attendre l'image suivante.
    }
}

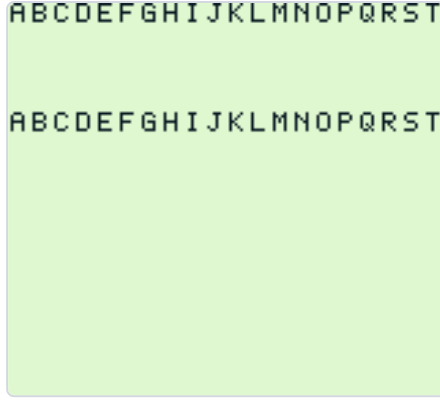
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

A à T deux fois : sur la ligne 0 et sur la ligne 5.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ la ligne 0 : A à T

✓ la ligne 5 : A à T

Ensuite : **0.5. La même ligne, avec une boucle for** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++